

Electronics Design with VHDL.

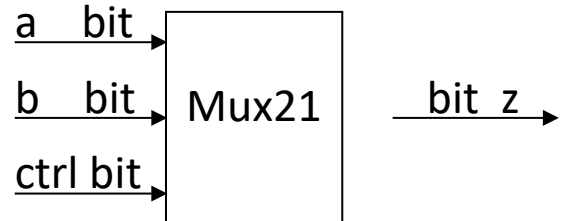
BASIC UNIT (V.0)

Department of Electronic. Área: Tecnología Electrónica

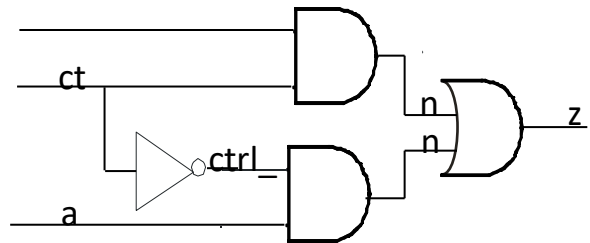
University of Alcalá.

01/09/2024

1: Architecture of Mux21 in algorithmic style



2: Architecture Mux21 in dataflow style



3: Structural architecture of Mux21

4: Given the following description in VHDL, determine the coding style

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;
```

entity adder is

```
    Port ( A : in  STD_LOGIC_VECTOR(3 downto 0);  
          B : in  STD_LOGIC_VECTOR(3 downto 0);  
          SUM : out STD_LOGIC_VECTOR(4 downto 0));
```

end adder;

architecture behavioral of adder is

begin

```
    process(A, B)
```

```
    begin
```

```
        SUM <= ("0" & A) + ("0" & B); -- Concatenamos un bit para evitar overflow
```

```
    end process;
```

end behavioral;

5: Determine the type of description used in the following VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity and_gate is
  Port ( A : in STD_LOGIC;
        B : in STD_LOGIC;
        Y : out STD_LOGIC);
end and_gate;
architecture dataflow of and_gate is
begin
  Y <= A AND B;
end dataflow;
```

6: Design a 2-bit comparator that determines whether two 2-bit numbers are equal, less, or larger using Algorithmic style.

7: Design a 4-bit adder that calculates the sum of two 4-bit numbers using the Dataflow style.

8: Design a 4-to-1 multiplexer using basic logic gates, in structural style.

9: Implements a Flip-Flop type D sensitive to the watch's rise edge, with asynchronous reset input using Algorithmic style.

10: Implement a 2-to-4 decoder using Dataflow style.

11: Design a two-input OR circuit (A, B) using only NAND gates in structural style. Use the logical expression of OR using NAND: $A+B=((A \cdot A) \cdot (B \cdot B))$ $A+B=((A \cdot A) \cdot (B \cdot B))$

12: Implement a 1-bit Full Adder using two Half Adders and an OR gate. Half Adders must add two input bits and provide one addition bit and one carry bit. The Full Adder adds three bits (A, B, and Carry-in) and generates a sum and a carry.

13: Design a 4-bit binary counter that increases its value on each rising edge of the clock. When you reach the maximum value (1111), you will need to reset to 0000.

14: Design a 4-bit adder that takes two 4-bit inputs (A and B) and returns a summed 4-bit output (Sum) and a carry bit (Carry).

Simulation models

15: Create a VHDL design that implements basic logic gates like AND, OR, and NOT. Design a testbench to simulate the behavior.

16: Design a simple half-adder adds two single bits and produces a sum and carry output.

17: Design a sequential logic element, the D flip-flop, captures the input on a rising clock edge.

18: Design a simple 4-bit counter that increments on each rising clock edge.